Group 27 Progress Report: Early-Game Win Prediction in League of Legends

Krish Panchal, Savvy Liu, Peter Li
{panchk5,liu1693,li227}@mcmaster.ca

1 Introduction

Our project estimates a team's chance of winning a League of Legends match from its position at ten minutes. We use both teams' economy, experience and combat statistics to predict the final result, with `blueWins=1` for a blue victory and 0 for a red victory. The aim is to understand how strongly early advantages predict the winner in historical matches.

The proposal considered logistic regression, support vector machines (SVMs), tree ensembles and neural networks. We have now run simple baselines, five logistic feature configurations and two random forests on fixed training and validation splits. We test whether objectives and vision improve predictions from economy and combat statistics, and whether forests improve on logistic regression. We measure both classification performance and probability error. The test set is reserved for the final evaluation.

2 Related Work

Junior and Campelo (2023) compare methods at several stages of League matches and report useful early-game results with logistic regression and gradient boosting. We include logistic regression as a simple reference for our feature comparisons. Their checkpoints use a percentage of eventual game duration; ours uses the dataset's fixed ten-minute snapshot, so the reported scores are not directly comparable.

Kim et al. (2020) study confidence calibration for League winner prediction. Correct win/loss predictions can still come with misleading confidence, so we also evaluate the predicted probabilities. We do not reproduce their uncertainty-aware neural loss. Guo et al. (2017) investigate temperature scaling for neural networks, a possible later calibration method. Its usefulness on our data would need to be tested.

Other League prediction work uses information that our dataset lacks. Silva et al. (2018) use recurrent networks to predict competitive match outcomes as games progress. Our single snapshot cannot capture those sequences. Do et al. (2021) predict ranked outcomes before gameplay using player–champion experience. Adding that context would require a different dataset. Both studies suggest directions for extending the project, although their tasks differ from our ten-minute prediction problem.

Niculescu-Mizil and Caruana (2005) examine probability quality across supervised methods. We report Brier score, log loss and reliability diagrams alongside accuracy and ROC-AUC. For the nonlinear comparison, we use random forests (Breiman, 2001) to learn interactions among the same inputs used by logistic regression.

3 Dataset

3.1 Source and contents

We use Yi Lan Ma's *League of Legends Diamond Ranked Games (10 min)*, Version 1 (Ma, 2020). The uploader attributes collection to Riot's API and describes high Diamond I to Master ranked solo-queue games. The CSV has no player-rank field, so we rely on the uploader for that description. Each row contains both teams' statistics and the final match result. The labels were supplied with the data; no manual annotation was needed.

The dataset contains 9,879 distinct matches and 40 columns: 38 source predictors, `gameId`, and `blueWins`. Blue wins 4,930 matches (49.90%); red wins 4,949 (50.10%). We found no missing cells, duplicate match IDs, duplicate complete predictor rows or constant columns. The included CSV is approximately 1.45 MB. Its SHA-256 hash is saved with the results to identify the exact file used.

Training feature	Mean	Std. dev.
Gold difference	-0.78	2431.96
XP difference	-53.66	1907.12
Kill difference	0.02	4.15
Wards placed difference	-0.09	26.04

Table 1: Selected blue-minus-red features in the 6,915 training matches, before scaling. The results include the full eleven-feature summary. The ward-count spread is one reason to investigate vision measurements.

3.2 Checks and preprocessing

Several columns repeat information. We checked that the supplied gold and XP differences equal blue totals minus red totals, that blue kills equal red deaths and vice versa, and that each team's gold and lane CS per minute equal the corresponding total divided by ten. These relationships let us remove exact redundancies when constructing the inputs. We exclude the match ID from the predictors.

We kept every row and applied no imputation, clipping or augmentation. Training-feature summaries include counts beyond three training standard deviations. Some ward counts are unusually large. We kept them because the available data do not let us distinguish errors from genuine observations, and will investigate them before the final experiment. Logistic scaling uses training rows only; forests use unscaled inputs.

3.3 Provenance, use and limitations

The CSV omits dates, region, patch versions and player identifiers. As a result, we cannot test performance across time or patches, check for shared players across splits, or claim current-game performance. We also rely on the uploader's ten-minute description because raw timeline timestamps are unavailable. Inputs are the supplied snapshot statistics; the final outcome is used only as the target.

We checked the published policies in response to the TA's data-use question. The publisher lists CC0 (Ma, 2020), and Riot's API terms allow documented purposes, including the Developer Portal's explicit allowance for personal research (Riot Games, 2013, 2026). We rely on that research allowance for this offline, non-commercial study. The policies do not specifically discuss model training, and we have not independently verified the uploader's licensing authority. Our experiments use the static aggregate CSV without player identifiers or live-game assistance. The separate API feasibility data are excluded.

4 Features

For each selected quantity q, we calculate $d_q = q_{\text{blue}} - q_{\text{red}}$. The seven core inputs are differences in total gold, total XP, average level, lane minions killed, jungle minions killed, kills and assists. Positive values mean blue has more of that quantity. We omit deaths because they are determined by kills, and omit fixed-time rate columns that repeat totals. Each quantity enters the model as a difference; the corresponding team totals are left out.

We then add dragon and Rift Herald count differences as the objective group, and wards placed and wards destroyed differences as the vision group. This gives one input for gold alone, seven for core, nine for core plus either group, and eleven for both. Testing the groups separately helps identify which addition contributes to a change in performance. We chose these groups from the proposal and the redundancy checks before comparing validation scores.

This compact representation loses absolute-state information: the same gold lead can occur in matches with different total economies. We plan to compare it with side totals. All current inputs are numerical; there are no embeddings or one-hot champion/item encodings. Gold, XP and level also remain correlated. Their coefficients describe conditional associations and do not identify causal effects.

5 Implementation

5.1 Baselines and logistic regression

The majority baseline always predicts red, the more common training label: 3,464 red wins versus 3,451 blue wins. For probability metrics, a separate baseline assigns every match the training blue-win frequency, $3451/6915 = 0.49906$.

Gold-only logistic regression measures how much can be predicted from the gold lead alone. We compare it with core, core plus objectives, core plus vision, and core plus both. All five fits use the same match IDs and optimization settings, so changes in their scores reflect the different input sets.

For a standardized input vector z_i, logistic regression predicts

$$p_i = \frac{1}{1 + \exp[-(b + w^\top z_i)]}. \tag{1}$$

With equally weighted training matches, the scikit-learn L2 objective is

$$J(w,b) = -\frac{1}{n}\sum_{i=1}^{n}\big[y_i \log p_i + (1-y_i)\log(1-p_i)\big] + \frac{\|w\|_2^2}{2nC}. \quad (2)$$

The intercept is not penalized. We set $C = 1$ and use L-BFGS, a limited-memory quasi-Newton method (scikit-learn developers, 2025). We chose it over the proposal's mini-batch SGD because the small, dense input set is straightforward to fit with a deterministic solver and needs no learning-rate schedule. Tolerance is 10^{-6}, with a limit of 2,000 iterations. All five fits converged in 7–22 iterations without convergence warnings. We have not tuned hyperparameters, class weights or the decision threshold.

5.2 Random forests

We fit forests on the seven core inputs and the eleven expanded inputs, using the same split and validation match order as logistic regression. Each forest has 500 bootstrap trees, maximum depth 8, minimum 10 samples per leaf, square-root feature subsampling, Gini impurity and seed 42. We fixed these settings before viewing the forest scores. The depth and leaf-size limits control complexity; tuning remains future work.

Each tree chooses threshold splits that greedily reduce Gini impurity. It does not optimize the logistic loss through gradients. The forest averages its trees' leaf class proportions to obtain a win probability. We use unscaled differences because threshold splits do not require standardization. Comparing each forest with the matching logistic model tests whether nonlinear interactions help on the same inputs and matches.

5.3 Reproducible pipeline

Each logistic pipeline contains a `StandardScaler` followed by `LogisticRegression`. The scaler learns feature means and population standard deviations from training rows and applies them to validation rows. This controls the effect of feature units on the L2 penalty. Both model families predict a blue win at a probability of at least 0.5.

The scripts save the source hash, split assignments, feature lists, settings, package versions, diagnostics and validation predictions. We used Python 3.12.14 with pandas 2.2.3, NumPy 2.0.2, scikit-learn 1.6.1 and Matplotlib 3.9.3. The package includes scripts for both model families, an independent check of the saved metrics, and an explanatory notebook. Runs reuse the saved splits and do not evaluate test performance.

6 Results and Evaluation

6.1 Protocol and metrics

Using stratification and seed 42, we split the matches into 6,915 training, 1,482 validation and 1,482 test rows (approximately 70/15/15). Validation contains 740 blue wins and 742 red wins. Stratification preserves the class balance without over/undersampling. Dataset-wide checks cover structure and class counts; all learned preprocessing and fitted models use training rows only. The missing dates and player IDs prevent a chronological or player-separated split.

All reported scores are preliminary validation results. We will evaluate the test set after model selection; cross-validation has not yet been used. Accuracy, precision, recall and F1 use blue as the positive class and a 0.5 threshold. ROC-AUC measures probability ranking. Brier score is $n^{-1}\sum_i (p_i - y_i)^2$, and log loss is mean unregularized binary cross-entropy on held-out matches. Lower Brier and log loss are better. Precision is set to zero when a model predicts no blue wins. We report no probability metrics for the hard majority classifier.

6.2 Preliminary findings

Gold alone reaches 71.05% accuracy, compared with 50.07% for the majority baseline (Table 2). Core reaches 71.93%. Adding objectives gives the highest accuracy, 72.27%, and lowers log loss from 0.54331 to 0.53969. Vision alone leaves accuracy unchanged and slightly worsens probability error. Adding both groups gives 72.13% accuracy and log loss 0.53990. Objectives contribute more than the two vision counts on this split, although the gains are small.

The combined extension gets 1,069 matches correct, just three more than core's 1,066. Its confusion matrix has 518 correctly predicted red wins, 551 correctly predicted blue wins, 224 false blue predictions and 189 missed blue wins. Core has 219 false blue predictions and 197 missed blue wins. The extension catches more blue victories at the cost of more false blue predictions.

Model	Inputs	Acc.	Prec.	Recall	F1	AUC	Brier	Log loss
Majority class	0	0.5007	0.0000	0.0000	0.0000	–	–	–
Training win rate	0	0.5007	0.0000	0.0000	0.0000	0.5000	0.2500	0.6931
Gold-only LR	1	0.7105	0.7043	0.7243	0.7142	0.7926	0.1861	0.5496
Core LR	7	0.7193	0.7126	0.7338	0.7230	0.7985	0.1836	0.5433
Core + objectives LR	9	0.7227	0.7123	0.7459	0.7287	0.8006	0.1828	0.5397
Core + vision LR	9	0.7193	0.7126	0.7338	0.7230	0.7984	0.1836	0.5434
Core + objectives + vision LR	11	0.7213	0.7110	0.7446	0.7274	0.8004	0.1828	0.5399
Core RF	7	0.7179	0.7048	0.7486	0.7261	0.7965	0.1844	0.5457
Core + objectives + vision RF	11	0.7186	0.7073	0.7446	0.7255	0.8004	0.1828	0.5401

Table 2: Preliminary validation results on the same 1,482 matches. LR is L2 logistic regression; RF is random forest. Blue is the positive class. Lower Brier and log loss are better. Probability baseline scores use the training win rate. No test scores are reported.

Core and expanded forest accuracies are 71.79% and 71.86%, slightly below the matching logistic models' 71.93% and 72.13%. Expanded forest log loss is 0.54012, close to logistic regression's 0.53990. The paired-bootstrap accuracy intervals for forest minus logistic are $[-1.08, 0.74]$ percentage points for core and $[-1.28, 0.74]$ for expanded inputs. Both span zero, so this comparison gives no clear advantage to either family. Different settings or inputs still need testing.

For the planned combined-extension minus core comparison, we resampled validation matches in pairs 2,000 times with seed 42, keeping each match's predictions together. The accuracy gain is +0.20 percentage points, with a percentile 95% interval of $[-1.21, 1.69]$ points. The log-loss difference is -0.00342 with interval $[-0.00980, 0.00291]$, and the Brier difference is -0.00078 with interval $[-0.00339, 0.00188]$. All three intervals include zero. The bootstrap holds the fitted models fixed: it omits training and split variability, possible dependence between matches involving the same players, and adjustment for the multiple comparisons inspected.

The reliability curves broadly follow the diagonal but have visible deviations (Figure 1). Binning can hide local errors, especially where few matches fall in a bin, so these plots do not establish full calibration. Gold already captures much of the predictive signal in our inputs. The small gains from extra features, together with the bootstrap intervals, make a firm ranking premature.

7 Feedback and Plans

Hongyi Zhang's October 9 Teams feedback asked for a formal dataset citation, basic feature and class statistics, evidence about training use, baselines and a system strategy. We added the citation, sample and class counts, and feature statistics in Table 1. Section 3.3 explains the published data-use policies we rely on. The implemented pipeline now includes majority, constant-probability and gold-only baselines, followed by the logistic and forest comparisons. Proposal feedback also asked why SVMs and neural networks suit this task and flagged a nested ZIP. We explain the model choices below; the submission contains the report, code and data in one archive with no nested ZIP.

An RBF SVM is worth testing because it can fit nonlinear boundaries in this small numerical feature set. Its probability estimates would need separate evaluation. A small neural network could learn interactions between economy and objectives, though overfitting and calibration would be concerns on this dataset. We started with a forest as the nonlinear comparison. Its similar scores give us little reason to add complexity without testing what it improves.

Our next priority is to understand the small feature gains and the remaining errors. We will check the unusual vision measurements and compare representations before deciding whether to keep those inputs. The current result concerns two ward counts in this dataset; it says little about the broader importance of vision in League.

By **November 17**, Peter will compare regularization and forest complexity with stratified cross-validation on training data. By **November 24**, Savvy will check feature representations and vision quality, and document the separate fresh-data extension. By **December 1**, Krish will lead calibration and error analysis and write up the limitations. Any calibrator will be fitted using training folds.

By **December 7**, we will review and freeze the model, feature and calibration choices. Peter will then run the held-out test evaluation once, and

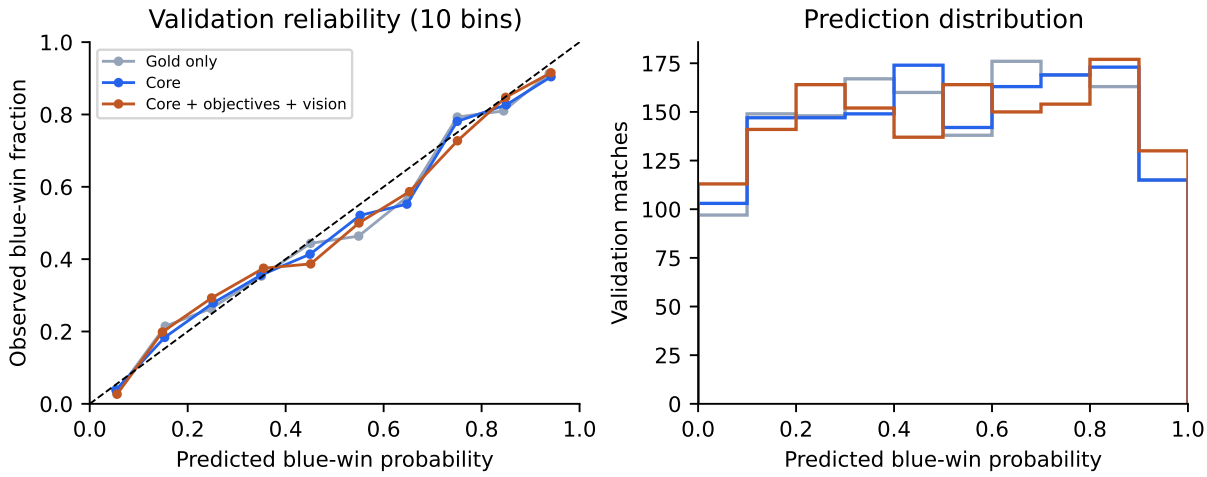


Figure 1: Validation reliability and probability distributions for gold-only, core and combined-extension logistic models. Points summarize ten equal-width bins. The diagonal marks agreement between predicted probabilities and observed blue-win frequencies; departures include sampling noise.

Savvy will verify reproducibility. For **December 10**, Krish will assemble the final report, Savvy will check the package, and Peter will audit the reported results. All three members will review the submission.

We will keep using the validation set for development, recognizing that repeated comparisons can make its scores optimistic. Fresh API data and champion/item features are a separate extension. The historical-data experiment can be reproduced without API collection.

Team Contributions

The original proposal was divided among all three members. **Savvy Liu** implemented the preprocessing, logistic and forest experiments, and evaluation checks, and wrote this report with the AI assistance described below. **Peter Li** reviewed and approved the progress-report work, in addition to contributing to the proposal. **Krish Panchal** also contributed to the proposal and reviewed and approved the progress-report work.

Generative AI Disclosure

We used OpenAI Codex (session-recorded model `gpt-6.1-sol`) for source retrieval, planning, code generation, debugging, interpretation, drafting and editing. We also used an AI review supplied to the team. The results come from running the included code locally and checking its reproducibility. Through this November 10 editing pass, the project chat contains **23 user queries**, including clarification responses and repository/API-setup discussions. The syllabus estimate is $23 \times 4.32 = 99.36$ **g CO_2**. This approximates the counted chat activity; the query count behind the separate AI review is unknown and excluded. OpenAI did not expose the hosted hardware, inference time or compute region. Local experiment runtimes measure code execution and cannot supply those missing inference details.

References

Leo Breiman. 2001. Random forests. *Machine Learning*, 45:5–32.

Tiffany D. Do, Seong Ioi Wang, Dylan S. Yu, Matthew G. McMillian, and Ryan P. McMahan. 2021. Using Machine Learning to Predict Game Outcomes Based on Player-Champion Experience in League of Legends. In *The 16th International Conference on the Foundations of Digital Games (FDG) 2021*, pages 1–5.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR.

Jailson B. S. Junior and Claudio E. C. Campelo. 2023. League of Legends: Real-Time Result Prediction. *Preprint*, arXiv:2309.02449.

Dong-Hee Kim, Changwoo Lee, and Ki-Seok Chung. 2020. A Confidence-Calibrated MOBA Game Winner Predictor. In *2020 IEEE Conference on Games (CoG)*, pages 622–625.

Yi Lan Ma. 2020. League of Legends Diamond Ranked Games (10 min). Kaggle dataset, Version 1.

Alexandru Niculescu-Mizil and Rich Caruana. 2005. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning*, pages 625–632.

Riot Games. 2013. API Terms and Conditions. Licensed Uses and Restrictions. Accessed November 10, 2026.

Riot Games. 2026. Developer Portal: Personal API Keys. Undated documentation. Accessed November 10, 2026.

scikit-learn developers. 2025. Logistic regression: User guide, version 1.6.1. Accessed November 10, 2026.

Antonio Luis Cardoso Silva, Gisele Lobo Pappa, and Luiz Chaimowicz. 2018. Continuous Outcome Prediction of League of Legends Competitive Matches Using Recurrent Neural Networks. In *Proceedings of SBGames 2018, Computing Track, Short Papers*.